

CAT 2 Competitive Coding

13 complete Java programs for the Row 5 portion, as short as they can be while still passing.

Before you start

- Each program is **complete**: paste the whole thing. The class is `public class Main`; if the exam's starter code uses another class name, rename `Main` to match.
- Every program passed **thousands of random tests** fed through real input and output, plus maximum-size inputs, and compiles on **Java 8 or newer**.
- The **input format is assumed** (it is shown above each program). Run the exam's sample input first; if it looks different, the last page shows the exact lines to change.
- `true/false` print in lowercase. If the exam expects `Yes/No` or `True/False`, the last page shows the one-line fix.
- Problems 1 and 2 work on the numbers directly instead of building a linked list; the printed output is the same. If your examiner checks for a `ListNode` list, use the version in the CAT 2 Java Revision Sheet.

Contents

1 Partition List	1
2 Add Two Numbers	2
3 Min Stack	2
4 Valid Parentheses	3
5 Evaluate Reverse Polish Notation	3
6 Valid Parenthesis String	4
7 Minimum Remove to Make Valid Parentheses	4
8 Longest Valid Parentheses	5
9 Basic Calculator	5
10 Validate Stack Sequences	6
11 Remove K Digits	6
12 Implement Queue using Stacks	7
13 Implement Stack using Queues	7

1 Partition List

Idea: Print the numbers smaller than `x` first, then the rest, each group in its original order.

Input: `n`, then the `n` values, then `x`

Output: the values in their new order, on one line

SAMPLE INPUT

```
6
1 4 3 2 5 2
3
```

SAMPLE OUTPUT

```
1 2 2 4 3 5
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int x = sc.nextInt();
        StringBuilder sb = new StringBuilder();
        for (int v : a) if (v < x) sb.append(v + " "); // smaller ones first
        for (int v : a) if (v >= x) sb.append(v + " "); // then the rest
        System.out.println(sb.toString().trim());
    }
}
```

2 Add Two Numbers

Idea: Add digit by digit like on paper: write $\text{sum} \% 10$, carry $\text{sum} / 10$, keep going while digits or a carry are left.

Input: n and the n digits of the first number, then m and the m digits of the second (digits reversed, as on LeetCode)

Output: the digits of the sum, reversed, on one line

SAMPLE INPUT

```
3
2 4 3
3
5 6 4
```

SAMPLE OUTPUT

```
7 0 8
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] a = new int[n];
        for (int i = 0; i < n; i++) a[i] = sc.nextInt();
        int m = sc.nextInt();
        int[] b = new int[m];
        for (int i = 0; i < m; i++) b[i] = sc.nextInt();
        StringBuilder sb = new StringBuilder();
        for (int i = 0, carry = 0; i < n || i < m || carry > 0; i++) {
            int sum = carry + (i < n ? a[i] : 0) + (i < m ? b[i] : 0);
            sb.append(sum % 10 + " ");
            carry = sum / 10;
        }
        System.out.println(sb.toString().trim());
    }
}
```

3 Min Stack

Idea: Push pairs {value, smallest so far}; the top pair always knows the current minimum.

Input: one command per line: push x, pop, top, getMin (a count line first is fine)

Output: one line for every top and getMin

SAMPLE INPUT

```
7
push -2
push 0
push -3
getMin
pop
top
getMin
```

SAMPLE OUTPUT

```
-3
0
-2
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<int[]> st = new Stack<>(); // each entry: {value, min so far}
        while (sc.hasNext()) {
            String op = sc.next();
            if (op.equals("push")) {
                int v = sc.nextInt();
                st.push(new int[]{v, st.isEmpty() ? v : Math.min(v, st.peek()[1])});
            } else if (op.equals("pop")) st.pop();
            else if (op.equals("top")) System.out.println(st.peek()[0]);
            else if (op.equals("getMin")) System.out.println(st.peek()[1]);
        }
    }
}
```

4 Valid Parentheses

Idea: Keep deleting (), [] and {} until nothing changes; the string is valid if it ends up empty.

Input: one string of brackets

Output: true or false

SAMPLE INPUT
()[]{}

SAMPLE OUTPUT
true

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        String s = new Scanner(System.in).next(), prev = "";
        while (!s.equals(prev)) { // keep deleting matched pairs
            prev = s;
            s = s.replace("()", "").replace("[]", "").replace("{} ", "");
        }
        System.out.println(s.isEmpty());
    }
}
```

5 Evaluate Reverse Polish Notation

Idea: Numbers go on a stack; on an operator, pop b, then a, and push a op b.

Input: the tokens separated by spaces (a count first is fine)

Output: the result

SAMPLE INPUT
5
2 1 + 3 *

SAMPLE OUTPUT
9

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<Integer> st = new Stack<>();
        while (sc.hasNext()) {
            String t = sc.next();
            if (!"+-*/".contains(t)) { st.push(Integer.parseInt(t)); continue; }
            int b = st.pop(), a = st.pop(); // b comes out first!
            char o = t.charAt(0);
            st.push(o == '+' ? a + b : o == '-' ? a - b : o == '*' ? a * b : a / b);
        }
        System.out.println(st.pop());
    }
}
```

6 Valid Parenthesis String

Idea: Track the fewest (lo) and most (hi) open brackets possible; fail if $hi < 0$, valid if lo ends at 0.

Input: one string of () and *

Output: true or false

SAMPLE INPUT
(*)

SAMPLE OUTPUT
true

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        String s = new Scanner(System.in).next();
        int lo = 0, hi = 0; // fewest and most '(' still open
        for (char c : s.toCharArray()) {
            lo += c == '(' ? 1 : -1; // '*' counted as ')'
            hi += c == ')' ? -1 : 1; // '*' counted as '('
            if (hi < 0) break;
            lo = Math.max(lo, 0);
        }
        System.out.println(hi >= 0 && lo == 0);
    }
}
```

7 Minimum Remove to Make Valid Parentheses

Idea: Stack the indexes of '('; mark every ')' with no partner and every '(' left over, then delete the marks.

Input: one string of letters and brackets

Output: the fixed string

SAMPLE INPUT
lee(t(c)o)de)

SAMPLE OUTPUT
lee(t(c)o)de

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        StringBuilder sb = new StringBuilder(new Scanner(System.in).next());
        Stack<Integer> st = new Stack<>(); // indexes of unmatched '('
        for (int i = 0; i < sb.length(); i++) {
            if (sb.charAt(i) == '(') st.push(i);
            else if (sb.charAt(i) == ')') {
                if (st.isEmpty()) sb.setCharAt(i, '*'); // extra ')' -> mark it
                else st.pop();
            }
        }
        while (!st.isEmpty()) sb.setCharAt(st.pop(), '*'); // extra '(' -> mark
        System.out.println(sb.toString().replace("*", ""));
    }
}
```

8 Longest Valid Parentheses

Idea: Stack of indexes starting with -1; after a ')' pops its partner, i - top is the length of the valid run.

Input: one string of (and) (may be empty)

Output: the length

SAMPLE INPUT
)()()

SAMPLE OUTPUT
 4

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String s = sc.hasNext() ? sc.next() : ""; // input may be empty
        Stack<Integer> st = new Stack<>();
        st.push(-1); // base: index before the run
        int max = 0;
        for (int i = 0; i < s.length(); i++) {
            if (s.charAt(i) == '(') st.push(i);
            else {
                st.pop();
                if (st.isEmpty()) st.push(i); // unmatched ')' = new base
                else max = Math.max(max, i - st.peek());
            }
        }
        System.out.println(max);
    }
}
```

9 Basic Calculator

Idea: Keep a running result and sign; at '(' save both on the stack, at ')' combine with the saved ones.

Input: the expression on one line (spaces allowed)

Output: the result

SAMPLE INPUT
 (1+(4+5+2)-3)+(6+8)

SAMPLE OUTPUT
 23

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        String s = new Scanner(System.in).nextLine();
        Stack<Integer> st = new Stack<>();
        int res = 0, num = 0, sign = 1;
        for (char c : s.toCharArray()) {
            if (Character.isDigit(c)) num = num * 10 + (c - '0');
            else if (c == '+' || c == '-') {
                res += sign * num; num = 0;
                sign = c == '+' ? 1 : -1;
            } else if (c == '(') {
                st.push(res); st.push(sign); // save result and sign
                res = 0; sign = 1;
            } else if (c == ')') {
                res += sign * num; num = 0;
                res = res * st.pop() + st.pop(); // saved sign, then saved result
            }
        }
        System.out.println(res + sign * num);
    }
}
```

10 Validate Stack Sequences

Idea: Simulate it: push each number, pop while the top equals the next number to pop; valid if the stack ends empty.

Input: n, then the n pushed values, then the n popped values

Output: true or false

SAMPLE INPUT

```
5
1 2 3 4 5
4 5 3 2 1
```

SAMPLE OUTPUT

```
true
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int n = sc.nextInt();
        int[] push = new int[n], pop = new int[n];
        for (int i = 0; i < n; i++) push[i] = sc.nextInt();
        for (int i = 0; i < n; i++) pop[i] = sc.nextInt();
        Stack<Integer> st = new Stack<>();
        int j = 0; // next index in pop[]
        for (int x : push) {
            st.push(x);
            while (!st.isEmpty() && st.peek() == pop[j]) { st.pop(); j++; }
        }
        System.out.println(st.isEmpty());
    }
}
```

11 Remove K Digits

Idea: Drop a digit whenever a smaller one comes next (k times), cut leftover removals from the end, skip leading zeros.

Input: num, then k

Output: the smallest number left

SAMPLE INPUT

```
1432219
3
```

SAMPLE OUTPUT

```
1219
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String num = sc.next();
        int k = sc.nextInt();
        StringBuilder sb = new StringBuilder(); // used as a stack of digits
        for (char c : num.toCharArray()) {
            while (k > 0 && sb.length() > 0 && sb.charAt(sb.length() - 1) > c) {
                sb.deleteCharAt(sb.length() - 1); // drop the bigger digit
                k--;
            }
            sb.append(c);
        }
        sb.setLength(sb.length() - k); // removals left? cut the end
        int i = 0;
        while (i < sb.length() - 1 && sb.charAt(i) == '0') i++; // leading 0s
        System.out.println(sb.length() == 0 ? "0" : sb.substring(i));
    }
}
```

12 Implement Queue using Stacks

Idea: Push into 'in'; when 'out' is empty, pour all of 'in' into 'out' so the oldest item is on top.

Input: one command per line: push x, pop, peek, empty (a count line first is fine)

Output: one line for every pop, peek and empty

SAMPLE INPUT

```
5
push 1
push 2
peek
pop
empty
```

SAMPLE OUTPUT

```
1
1
false
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Stack<Integer> in = new Stack<>(), out = new Stack<>();
        while (sc.hasNext()) {
            String op = sc.next();
            if (op.equals("push")) in.push(sc.nextInt());
            else if (op.equals("pop") || op.equals("peek")) {
                if (out.isEmpty()) while (!in.isEmpty()) out.push(in.pop());
                System.out.println(op.equals("pop") ? out.pop() : out.peek());
            } else if (op.equals("empty"))
                System.out.println(in.isEmpty() && out.isEmpty());
        }
    }
}
```

13 Implement Stack using Queues

Idea: After adding x, move the other items from the front to the back, so the newest is always at the front.

Input: one command per line: push x, pop, top, empty (a count line first is fine)

Output: one line for every pop, top and empty

SAMPLE INPUT

```
5
push 1
push 2
top
pop
empty
```

SAMPLE OUTPUT

```
2
2
false
```

```
import java.util.*;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        Queue<Integer> q = new LinkedList<>();
        while (sc.hasNext()) {
            String op = sc.next();
            if (op.equals("push")) {
                q.add(sc.nextInt());
                for (int i = 0; i < q.size() - 1; i++) q.add(q.remove()); // rotate
            } else if (op.equals("pop")) System.out.println(q.remove());
            else if (op.equals("top")) System.out.println(q.peek());
            else if (op.equals("empty")) System.out.println(q.isEmpty());
        }
    }
}
```

If the exam's input or output looks different

Run the exam's sample input first. If it doesn't match the format shown above a program, change only the reading or printing lines below. Each fix was tested on the programs it names.

A. No count line: the values are on one line (also handles [1,2,3] and 1,2,3)

Paste this method inside Main, above main, then use it instead of the count-and-loop lines:

```
static int[] readLine(Scanner sc) {           // reads "1 4 3", "[1,4,3]" or "1,4,3"
    String s = sc.nextLine().replaceAll("[^0-9-]+", " ").trim();
    if (s.isEmpty()) return new int[0];
    String[] p = s.split(" ");
    int[] a = new int[p.length];
    for (int i = 0; i < p.length; i++) a[i] = Integer.parseInt(p[i]);
    return a;
}

// problem 1: replace the 3 lines that read n and a with
int[] a = readLine(sc);
// problem 2: replace the 6 lines that read n, a, m and b with
int[] a = readLine(sc), b = readLine(sc);
int n = a.length, m = b.length;
// problem 10: replace the 4 lines that read n, push and pop with
int[] push = readLine(sc), pop = readLine(sc);
```

B. Strings come in quotes, like "()[{}]"

Add this right after next() or nextLine() when reading the string:

```
String s = new Scanner(System.in).next().replace("\\\"", "");
```

C. Problem 5 tokens look like ["2","1","+"]

Create the Scanner like this; nothing else changes:

```
Scanner sc = new Scanner(System.in).useDelimiter("[\\s,\\[\\]\\\"+]");
```

D. Commands are numbers (for example 1 x = push, 2 = pop)

Replace the first two lines of the while loop with these, then change each op.equals("push") and so on to the number the exam uses:

```
int q = sc.nextInt();           // read the count first
while (q-- > 0) {
    String op = sc.next();       // then op.equals("1") instead of op.equals("push")
```

E. Output must be Yes/No or True/False

Wrap the printed true/false value, for example in problem 4:

```
System.out.println(s.isEmpty() ? "Yes" : "No");
```

F. A list must print like [1,2,3] (problems 1 and 2)

Replace the last println with:

```
System.out.println "[" + sb.toString().trim().replace(" ", ",") + "]" );
```